EDP-Aware Mapping of LLM Computation Blocks on Multicore Architectures

Overall good proposal. Needed bit more details on evaluation platform.

itya Dhodade
ience & Automation
stitute of Science
galuru, India
nd@iisc.ac.in

1st Prat
Computer Scie *n*
Indian Inst
Benga
pratika

8/10

for
neous thread scheduling, executing all transformer blocks under uniform core affinity and frequency settings. This approach ignores the heterogeneous nature of transformer workloads, where compute-bound tensor contractions coexist with memory-stalled operations. Consequently, current frameworks often face a suboptimal trade-off between latency and power consumption, leading to a degraded Energy–Delay Product (EDP). In this work, we propose a computation-aware block-to-core mapping strategy that categorizes transformer blocks based on their roofline characteristics. The proposed design functions by dynamically aligning compute-intensive kernels with high-performance cores while consolidating memory-bound operations onto energy-efficient execution paths. To realize this, the framework envisages the use of compatible operating system interfaces for frequency scaling and thread management. By exploiting producer–consumer locality through cache-aware affinity, the strategy aims to minimize data movement overhead. This approach is intended to be evaluated within the llama.cpp runtime to demonstrate that tailoring hardware resources to specific transformer workloads can achieve significant energy efficiency without compromising performance.

***Index Terms*—computation aware scheduling, energy delay product, inference on CPU, core mapping**

I. Introduction

Large language models (LLMs) are increasingly transitioning from cloud-hosted services to execution on local platforms. This shift is driven by growing demands for reduced inference latency and saving energy. However, enabling efficient local inference remains challenging, as modern LLMs impose computational and memory demands while CPUs in these environments lack the specialized tensor accelerators available in datacenter.

LLMs comprises of heterogeneous computation blocks with distinct performance characteristics. Compute intensive tensor contractions, such as GEMMs in attention and feed-forward layers, shows high operational intensity and scale with core frequency, whereas normalization, softmax, and element-wise operations are primarily memory-bound.

CPU-based LLM inference frameworks, such as *llama.cpp* and its GGML backend [9], employ largely homogeneous execution models. Transformer blocks are scheduled using static or round-robin thread assignment, with limited awareness of block-level computational characteristics. Consequently, compute and memory bound kernels execute under identical placement and frequency settings, leading to suboptimal performance–energy trade-offs. Additionally, default thread placement fails to exploit producer–consumer locality, resulting in unnecessary data movement and cache overheads.

Prior work in high-performance computing has demonstrated that coordinated tuning of thread affinity and dynamic voltage and frequency scaling (DVFS) [6] can reduce the Energy Delay Product (EDP) of parallel workloads on multicore systems. Frameworks such as GAAMP show that joint optimization of core placement, thread counts, and frequency selection can yield energy savings with minimal performance loss. While these techniques are well established for homogeneous workloads, their use in CPU-based LLM inference remains largely uniform and model-wide. Existing optimization efforts primarily focus on GPU-centric techniques or hardware acceleration, leaving computation-aware, block-level EDP optimization for transformer inference on multicore CPUs insufficiently explored.

II. Related Work

The optimization of Large Language Model (LLM) inference on general-purpose CPUs has gained traction as local execution becomes a priority for privacy and low-latency applications. Current research can be categorized into workload characterization, architectural mapping, and energy-efficient scheduling.

A. Phase-Wise Characterization

Understanding the computational bottleneck of LLMs is fundamental for optimizing resource allocation. The work in [1] and [2] identifies a clear difference in transformer workloads: the compute-bound phase and the memory-bound phase. This distinction is further formalized by the Energy-Roofline model [3], which introduces the concept of "energy-balance" as the ratio of operations to data movement per unit of energy. Our work builds on this by applying these roofline characteristics at the individual block level rather than just the phase level.

B. Hardware-Software Co-Optimization

Recent efforts have focused on CPU-specific instruction sets and memory hierarchies to bridge the performance gap with

GPUs. [4] demonstrated that the use of SIMD instructions in ARM many-core architectures, combined with component fusion, significantly improves throughput. For high-performance Intel systems, characterizations on HBM-equipped CPUs [5] reveal that memory bandwidth remains the primary constraint for autoregressive generation. Optimized libraries such as GGML [9] and general CPU-based transformer optimizations [10] provide the infrastructure for these enhancements; however, these works often focus on raw throughput and overlook the energy-delay trade-offs inherent in homogeneous scheduling.

C. EDP-Aware Scheduling and DVFS

The use of Dynamic Voltage and Frequency Scaling (DVFS) and core affinity to optimize the Energy-Delay Product (EDP) is well-established in high-performance computing but less so for LLMs. Frameworks have shown that joint optimization of thread placement and frequency selection can lead to substantial energy savings [6]. Recent offline models [7] confirm that predicting optimal energy states for inference workloads can reach an accuracy of $R^2 > 0.96$. Furthermore, the importance of thread pinning and core isolation to mitigate scheduler jitter is highlighted in [8], which is critical for maintaining predictable latency in real-time inference.

Our proposed strategy differentiates itself by dynamically mapping blocks to heterogeneous core groups within the `llama.cpp` runtime [9], specifically targeting the reduction of EDP through cache-aware affinity and per-block frequency scaling.

III. Brief Plan of Action

The proposed work will be carried out in four structured phases, progressing from workload characterization to system-level evaluation.

A. Transformer Block Characterization

We will first analyze the execution characteristics of transformer computation blocks commonly used in LLM inference, including attention projections, feed-forward networks, normalization layers, and element-wise operations. Using a roofline-based analysis informed by hardware performance counters, each block will be classified according to its dominant bottleneck (compute or memory-bound). This classification will guide subsequent mapping and frequency decisions.

B. Core and Frequency Domain Analysis

Modern multicore CPUs expose multiple frequency domains and shared cache hierarchies. We will change the core frequency and voltage level according to the requirement or the charactersitic of computation block of llm.

C. Computation-Aware Block-to-Core Mapping

Based on block characterization and hardware constraints, we will design a computation-aware mapping strategy that assigns transformer blocks to specific cores or core groups. Compute-intensive blocks will be preferentially mapped to high-frequency cores, while memory-bound blocks will be consolidated onto fewer, lower-frequency cores. Thread affinity will be chosen to preserve producer–consumer locality across consecutive blocks and minimize inter-core data movement.

D. Implementation and Evaluation

The proposed mapping will be implemented in userspace by extending an existing CPU-based LLM inference runtime. The implementation will provide mechanisms to control thread placement and execution frequency at the core or core-group level, which will treatment of compute-bound and memory-bound transformer blocks. These controls will be realized using operating-system-supported interfaces for affinity management and dynamic frequency scaling, without requiring kernel modifications or specialized hardware support. The methodology will be evaluated on a EDP calculated before and after applying mapping strategy.

IV. Final Goal

The final goal of this work is to quantitatively demonstrate that computation-aware block-to-core mapping can reduce the Energy Delay Product (EDP) of CPU-based LLM inference relative to homogeneous scheduling. Specifically, the proposed approach aims to achieve:

- A measurable reduction in EDP of 20–40% across multiple LLM architectures, without degrading model accuracy or functional correctness.
- A reduction in total energy consumption of inference workloads while maintaining inference latency within ±5% of baseline execution, or improving latency for compute-bound phases.

Performance will be evaluated using hardware performance counters and system-level energy measurements, with homogeneous thread scheduling and uniform frequency policies serving as the baseline for comparison.

References

[1] T. Husom et al., "Phase-wise Characterization of LLM Inference," *arXiv preprint arXiv:2512.03024*, 2024.

[2] "TokenPowerBench: A Framework for LLM Energy Profiling," 2025. [Online]. Available: https://arxiv.org/html/2512.03024v1

[3] J. Choi, "The Energy-Roofline Model: Defining Energy-Balance for Modern Processors," [Online]. Available: https://jeewhanchoi.github.io/publication/pdf/energy_roofline.pdf

[4] Y. Zhang, Q. Wu, and J. Li, "Characterizing and Optimizing Transformer Inference on ARM Many-core," in *Proc. ACM SIGPLAN Symp. Princ. Pract. Parallel Program. (PPoPP)*, 2022.

[5] S. Na et al., "Characterization of LLM Inference on Intel HBM-equipped CPUs," in *Proc. IEEE Int. Symp. Workload Characterization (IISWC)*, 2024.

[6] "Unified Formulations for DVFS and Task Migration in Multicore Systems," *ResearchGate*, 2026. [Online]. Available: https://www.researchgate.net/publication/260710832

[7] "Offline Scheduling Models for Energy-Optimal LLM Execution," *arXiv preprint arXiv:2407.04014*, 2024.

[8] "Overcoming OS-level Challenges for Low-Latency Inference," *Eunomia*, Feb. 2025. [Online]. Available: https://eunomia.dev/blog/2025/02/18/os-level-challenges/

[9] G. Gerganov, "GGML: A Tensor Library for Machine Learning," *GitHub*, 2024. [Online]. Available: https://github.com/ggerganov/ggml

[10] "Optimizing Inference Performance of Transformers on CPUs," *arXiv preprint arXiv:2102.06621*, 2021.